

Lección 7: Advanced Refactoring Patterns

Patrones Avanzados de Refactoring

Agenda

- Más Allá de Code Smells Básicos
 - Strategy Pattern
 - Factory Pattern
 - Observer Pattern
 - Command Pattern
 - Functional Programming Patterns
 - Composition over Inheritance
-

Progression of Refactoring (1/2)

BASIC
(nivel función)

- Magic Numbers
(números sin nombre)
- Duplicate Code
(código repetido)
- Long Methods
(métodos largos)
- Primitives
(obsesión primitivos)

ARCHITECTURAL
(nivel clase)

- God Objects
(clase hace todo)
- Feature Envy
(envidia funcionalidad)
- Inappropriate
Intimacy
(clases acopladas)

Progression of Refactoring (2/2)

SYSTEM-WIDE
(nivel sistema)

- Strategy Pattern
(encapsular algoritmos)
- Factory Pattern
(centralizar creación)
- Observer Pattern
(eventos desacoplados)
- Command Pattern
(operaciones como objetos)

Los patrones avanzados resuelven problemas arquitectónicos

1. Strategy Pattern

Problema: Switch statements que violan el Open/Closed Principle (principio abierto/cerrado)

```
// ❌ Switch statement anti-pattern
function calculateShipping(order, method: string) {
  switch (method) {
    case 'standard':
      return order.weight * 0.5 + 5.0
    case 'express':
      return order.weight * 1.0 + 15.0
    case 'overnight':
      return order.weight * 2.0 + 25.0
  }
}

// Múltiples switches en diferentes lugares
// Difícil añadir nuevos métodos de envío
```

Strategy Pattern: Solución

```
// ✅ Strategy Pattern
interface ShippingStrategy {
  calculateCost(order: Order): number
  getDescription(): string
}

class StandardShipping implements ShippingStrategy {
  calculateCost(order: Order) {
    return order.weight * 0.5 + 5.0
  }
  getDescription() {
    return 'Standard shipping'
  }
}

const STRATEGIES = {
  standard: new StandardShipping(),
  express: new ExpressShipping(),
}

// Fácil añadir nuevas strategies sin modificar código existente
```

2. Factory Pattern

Problema: Creación compleja de objetos dispersa

```
// ❌ Lógica de creación compleja repetida
if (type === 'stripe') {
  const processor = new StripeProcessor()
  processor.setApiKey(config.apiKey)
  processor.setWebhook(config.webhook)
  processor.enableTestMode(config.test)
  return processor
}
// Repetido para cada tipo de pago
```

Factory Pattern: Solución

```
// ✅ Factory Pattern
interface PaymentProcessor {
  processPayment(amount: number): Promise<PaymentResult>
}

class StripeFactory extends PaymentProcessorFactory {
  createProcessor(config: Config): PaymentProcessor {
    this.validateConfig(config)
    const processor = new StripeProcessor()
    processor.setApiKey(config.apiKey)
    return processor
  }
}

const FACTORIES = {
  stripe: new StripeFactory(),
  paypal: new PaypalFactory(),
}
```

3. Observer Pattern

Problema: Acoplamiento fuerte con eventos

```
// ❌ Cart conoce todos los observers
class ShoppingCart {
```

```

constructor(
    private analytics: AnalyticsService,
    private inventory: InventoryService,
    private notifications: NotificationService
) {}

addItem(product, quantity) {
    this.items.push({ product, quantity })
    this.analytics.track(...)
    this.inventory.reserve(...)
    this.notifications.show(...)
}
}

```

Observer Pattern: Solución

```

//  Observer Pattern – Bajo acoplamiento
interface CartEventObserver {
    onCartEvent(event: CartEvent): void
}

class ShoppingCart {
    private observers = new Set<CartEventObserver>()

    subscribe(observer: CartEventObserver) {
        this.observers.add(observer)
    }

    addItem(product, quantity) {
        this.items.push({ product, quantity })
        this.emit('item_added', { product, quantity })
    }

    private emit(type, data) {
        this.observers.forEach(o => o.onCartEvent({ type, data }))
    }
}

```

4. Command Pattern

Problema: Sin historial de operaciones ni undo

```
// ❌ Operaciones directas - no se puede deshacer
class CartManager {
    addItem(product, quantity) {
        this.cart.push({ product, quantity })
    }
    removeItem(index) {
        this.cart.splice(index, 1)
    }
    // ¡No hay forma de deshacer!
}
```

Command Pattern: Solución (1/2)

```
// ✅ Command Pattern - Operaciones como objetos
interface CartCommand {
    execute(): void
    undo(): void
}

class AddItemCommand implements CartCommand {
    execute() {
        this.cart.push(this.item)
        this.addedIndex = this.cart.length - 1
    }

    undo() {
        this.cart.splice(this.addedIndex, 1)
    }
}
```

Command Pattern: Solución (2/2)

```
class CartManager {
    private history: CartCommand[] = []

    executeCommand(command: CartCommand) {
        command.execute()
    }
}
```

```
        this.history.push(command)
    }

    undo() {
        this.history.pop()?.undo()
    }
}
```

5. Pure Functions

Problema: Side effects (efectos secundarios) y mutaciones

```
//  Impura - side effects
class DiscountCalculator {
    private appliedDiscounts: string[] = []

    calculateTotal(items) {
        let total = 0
        items.forEach(item => {
            total += item.price
            if (item.quantity >= 5) {
                this.appliedDiscounts.push(`bulk-${item.id}`) // iSide effect!

                total -= item.price * 0.1
            }
        })
        return total
    }
}
```

Pure Functions: Solución

```
//  Funciones puras - sin side effects
interface CalculationResult {
    subtotal: number
    discounts: DiscountApplication[]
    total: number
}

class DiscountCalculator {
```

```

    calculateTotal(items: CartItem[]): CalculationResult {
        const subtotal = this.calculateSubtotal(items)
        const discounts = this.calculateDiscounts(items)
        const total = subtotal - discounts.reduce((sum, d) => sum + d.amount, 0)

        return { subtotal, discounts, total }
        // Sin side effects, determinístico, testeable
    }
}

```

6. Composition over Inheritance

Problema: Jerarquías de herencia profundas

```

// ❌ Herencia profunda
abstract class PaymentMethod {
    abstract processPayment(amount: number): Promise<Result>
}

class CreditCardPayment extends PaymentMethod {}
class SecureCreditCard extends CreditCardPayment {}
class PremiumSecureCreditCard extends SecureCreditCard {}

// Difícil de extender, frágil, fuertemente acoplado

```

Composition: Solución

```

// ✅ Composition - flexible y testeable
interface PaymentProcessor {
    processPayment(amount: number): Promise<Result>
}

interface PaymentValidator {
    validate(data: PaymentData): ValidationResult
}

class CompositePaymentService {
    constructor(
        private processor: PaymentProcessor,
        private validators: PaymentValidator[],
    )

```



```
    private enhancers: PaymentEnhancer[]
  ) {}

  async processPayment(data: PaymentData) {
    // Validar
    for (const validator of this.validators) {
      validator.validate(data)
    }
    // Mejorar
    // Procesar
  }
}
```

Testing Advanced Patterns (Testeando Patrones Avanzados)

Strategy Pattern Testing:

```
describe('ShippingStrategy', () => {
  it('debería calcular costo correcto', () => {
    const strategy = new StandardShipping()
    expect(strategy.calculateCost(order)).toBe(6.25)
  })

  it('debería usar strategy correcta', () => {
    const calculator = new ShippingCalculator()
    expect(calculator.calculateCost(order, 'standard')).toBe(5.5)
    expect(calculator.calculateCost(order, 'express')).toBe(16.0)
  })
})
```

Métricas de Calidad: Cyclomatic Complexity

Complejidad Ciclomática: Número de caminos independientes en el código

```
// Cyclomatic Complexity = 4
function getDiscount(user, amount) {
  if (user.isPremium) {
    // +1
  }
}
```

```

    if (amount > 100) {
        // +1
        return 0.2
    }
    return 0.1
} else if (amount > 200) {
    // +1
    return 0.05
}
return 0
}

```

Regla: Alto (>10) = difícil testear, Bajo (<5) = mantenible

Métricas de Calidad: Coupling (Acoplamiento)

Coupling: Grado de dependencia entre módulos

```

// ❌ Alto acoplamiento - depende de 5 clases concretas
class Cart {
    constructor(
        private analytics: AnalyticsService,
        private inventory: InventoryService,
        private notifications: NotificationService
    ) {}
}

// ✅ Bajo acoplamiento - depende de abstracción
class Cart {
    private observers: CartObserver[] = []
}

```

Métricas de Calidad: Cohesion (Cohesión)

Cohesion: Grado en que elementos de un módulo pertenecen juntos

```

// ❌ Baja cohesión - hace cosas sin relación
class UserManager {
    createUser() {}
    calculateShipping() {} // ¿Shipping aquí?
}

```

```
    generateReport() {} // ¿Report aquí?
}

// ✅ Alta cohesión - todo relacionado con usuarios
class UserManager {
    createUser() {}
    updateUser() {}
    deleteUser() {}
}
```

Métricas de Calidad: Extensibility (Extensibilidad)

Extensibility: Facilidad para añadir funcionalidad sin modificar código

```
// ❌ Difícil - añadir método = modificar función
function calculateShipping(method: string) {
    switch (method) {
        case 'standard':
            return 5
        case 'express':
            return 15
    }
}

// ✅ Fácil - solo añadir nueva strategy
const STRATEGIES = {
    standard: new StandardShipping(),
    overnight: new OvernightShipping(), // Nueva sin modificar
}
```

Métricas de Calidad: Testability (Testeabilidad)

Testability: Facilidad para escribir tests unitarios

```
// ❌ Compleja - dependencias hard-coded
class OrderProcessor {
    process() {
        const db = new Database() // No se puede mockear
    }
}
```

```
    const api = new PaymentAPI()
  }
}

//  Simple - dependency injection
class OrderProcessor {
  constructor(
    private db: Database,
    private api: PaymentAPI
  ) {}
}
// Inyectar mocks en tests fácilmente
```

Beneficios: Antes vs Después

Antes de Patrones:

- Cyclomatic Complexity: 18
- Coupling (acoplamiento): Alto
- Cohesion (cohesión): Baja
- Extensibility (extensibilidad): Difícil
- Testability (testeabilidad): Compleja

Después de Patrones:

- Cyclomatic Complexity: 6 (-67%)
 - Coupling (acoplamiento): Bajo
 - Cohesion (cohesión): Alta
 - Extensibility (extensibilidad): Fácil
 - Testability (testeabilidad): Simple
-

SOLID Principles Compliance (Cumplimiento de Principios SOLID)

1. **Single Responsibility** (responsabilidad única): Una razón para cambiar
2. **Open/Closed** (abierto/cerrado): Abierto para extensión, cerrado para modificación
3. **Liskov Substitution** (sustitución de Liskov): Subtipos pueden reemplazar tipos

base

4. **Interface Segregation** (segregación de interfaces): Clientes dependen de métodos que usan
 5. **Dependency Inversion** (inversión de dependencias): Depender de abstracciones
-

Advertencias Anti-Pattern

```
// ❌ No sobre-ingeniería para casos simples
if (userType === 'premium') {
  discount = 0.1
} else {
  discount = 0.05
}
// Un simple if-else no necesita Strategy Pattern

// ❌ No usar patrones por usar patrones
function calculateTax(amount: number) {
  return amount * 0.08 // Simple, no necesita patrón
}
```

Cuándo Aplicar Patrones

Usar patrones cuando:

- Existen múltiples implementaciones similares
- El código cambia frecuentemente
- Se necesita flexibilidad y extensibilidad
- La complejidad justifica la abstracción

No usar patrones cuando:

- La implementación es simple y poco probable que cambie
 - Añade complejidad innecesaria
 - El equipo no entiende el patrón
-

Ejercicio 1: Explorar Strategy Pattern

Prompt:

Actúa como un desarrollador que estudia patrones de diseño en código real.

CONTEXT0: Strategy Pattern encapsula familias de algoritmos relacionados en clases separadas, haciendo que sean intercambiables. Elimina switch statements que violan Open/Closed Principle (abierto para extensión, cerrado para modificación). Cada strategy implementa la misma interfaz, permitiendo añadir nuevas strategies sin modificar código existente. Registry Pattern centraliza el acceso a strategies disponibles.

TAREA: Explora implementación real de Strategy Pattern en el proyecto.

ARCHIVO A ANALIZAR:

- src/shared/strategies/DiscountStrategy.ts (interface)
- src/shared/strategies/*.ts (implementaciones)

EXPLORACIÓN REQUIREMENTS:

1. Leer interface DiscountStrategy (métodos, tipos de retorno)
2. Identificar 3 implementaciones concretas (Standard, PremiumMember, Seasonal)
3. Analizar cómo cada strategy encapsula lógica diferente
4. Observar DiscountStrategyRegistry (cómo registra y accede strategies)
5. Ejecutar tests: `pnpm test DiscountStrategy`

PUNTOS DE OBSERVACIÓN:

- ¿Qué métodos define DiscountStrategy?
- ¿Cómo cada implementación calcula descuentos diferente?
- ¿Cómo se registran strategies en el Registry?
- ¿Cómo facilita añadir nuevas strategies sin modificar existentes?

VALIDACIÓN: 30+ tests deben pasar 

Aprende: Strategy Pattern elimina switches y facilita

extensión sin modificación

Ejercicio 2: Crear Nueva Strategy

Prompt:

Actúa como un desarrollador que practica Open/Closed Principle (OCP) con St

CONTEXTO: Open/Closed Principle establece que código debe estar abierto par extensión, cerrado para modificación. Strategy Pattern implementa OCP: añad nuevas strategies NO requiere modificar código existente, solo implementar interfaz. DiscountStrategy define contrato: calculateItemDiscount(), calculateOrderDiscount(), getStrategyName(). Cada strategy retorna DiscountResult {amount, description, applied}. Registry Pattern permite registrar strategies dinámicamente.

TAREA: Crea nueva LoyaltyPointsDiscountStrategy sin modificar código existe

STRATEGY SPECIFICATIONS:

- Nombre: LoyaltyPointsDiscountStrategy
- Interfaz: implements DiscountStrategy
- Constructor: recibe points: number
- Lógica: Si points $\geq 1000 \rightarrow$ 5% descuento en items, 0% descuento en orden
- getStrategyName(): retorna 'loyalty'

MÉTODO calculateItemDiscount:

- Si points ≥ 1000 : calcular 5% del totalValue (price $\times$ quantity)
- Retornar: {amount: descuento, description: '5% loyalty bonus', applied: t
- Si points < 1000 : {amount: 0, description: 'No loyalty discount', applied

MÉTODO calculateOrderDiscount:

- Siempre retornar: {amount: 0, description: 'No order discount', applied:

TEST REQUIREMENTS:

- Framework: Vitest
- Test data: item {price: 100}, quantity: 2, points: 1000
- Verificar: discount.amount === 10 (5% de 200)
- Verificar: discount.applied === true

REGISTRO (OPCIONAL):

- DiscountStrategyRegistry.register('loyalty', new LoyaltyPointsDiscountStr

ARCHIVOS:

- src/shared/strategies/LoyaltyPointsDiscountStrategy.ts
- src/shared/strategies/LoyaltyPointsDiscountStrategy.test.ts

VALIDACIÓN: ejecuta pnpm test LoyaltyPoints $\rightarrow$ test debe PASAR 

Aprende: Open/Closed Principle - extensión

sin modificación usando Strategy

Puntos Clave

1. **Strategy:** Elimina switches, habilita extensibilidad
2. **Factory:** Centraliza creación compleja de objetos
3. **Observer:** Desacopla eventos de handlers
4. **Command:** Habilita undo/redo y audit trails (pistas de auditoría)
5. **Pure Functions:** Elimina side effects
6. **Composition:** Reemplaza herencia compleja
7. **Usar sabiamente:** Los patrones resuelven problemas, no todo código los necesita